

AGU EE 304: EMBEDDED SYSTEMS

LAB 11: Autonomous Smart Vehicle with AI & Emergency Override

Week:	11
Instructor:	Dr. Khaled Hejja
Tech Stack:	C++, Python, Raspberry Pi

1.0 OBJECTIVES

By the conclusion of this session, we will be able to:

- Implement Persistent States:** Design a state machine where a single command (e.g., "move forward") initiates a continuous behavior.
- Autonomous Fail-Safe:** Code a high-priority monitoring loop that can override the current system state if sensor thresholds are breached (Standalone Obstacle Avoidance).
- Real-Time Data Fusion:** Combine high-level AI text/voice commands with low-level sensor distance data.
- Client-Server Mobile Control:** Interact with the Raspberry Pi 5 using a smartphone as the remote control interface.

2.0 BILL OF MATERIALS (BOM)

ITEM	DESCRIPTION	FUNCTION
MICROCONTROLLER	Raspberry Pi 5	The central processor for logic, AI handling, and networking.
SENSOR	HC-SR04 Ultrasonic Sensor	The "Eyes" for standalone obstacle detection.
STATUS LED (GREEN)	Visual Output	Indicates "Forward Motion" or "Safe" state.
STATUS LED (RED)	Visual Output	Indicates "Obstacle Detected" or "Emergency Avoidance."
RESISTORS	2x 330Ω	Current protection for LEDs.
SMARTPHONE	iOS/Android	Used to send voice-to-text commands to the Pi server.

3.0 HARDWARE CONNECTIONS (FULLY DETAILED)

HC-SR04 PIN	RASPBERRY PI 5 GPIO	NOTES
VCC	5V (Pin 2)	High power for the sensor.
GND	GND (Pin 6)	Common ground.
TRIG	GPIO 17 (Pin 11)	Digital Output trigger.
ECHO	GPIO 27 (Pin 13)	Digital Input listener.

LED COLOR	CONNECTION TO PI	NOTES
GREEN (SAFE)	GPIO 14 (Pin 8)	Signals vehicle is actively moving forward.
RED (WARNING)	GPIO 15 (Pin 10)	Signals proximity alarm/auto-stop.
GND (BOTH)	GND (Pin 9)	Connect common negative rails.

4.0 SOFTWARE IMPLEMENTATION

This code handles three things at once:

- Web Server (Core 0):** Listens for commands from your mobile phone.
- Sensor Watchdog (Core 1):** Continuously measures distance.
- Autonomous Controller:** Automatically switches state from forward to avoiding if an object is near.

Python

--- LAB 11: SMART VEHICLE CONTROL WITH STANDALONE AVOIDANCE ---

1. LIBRARY INCLUSION

```
import RPi.GPIO as GPIO # Imports the GPIO library to allow Python to control the Raspberry Pi's pins.
import time              # Imports the time library to handle delays and precise timing measurements.
import _thread           # Imports threading library to allow Core 0 and Core 1 to perform different tasks at once.
from flask import Flask, request, jsonify # Imports Flask components to create a web server reachable by mobile devices.
```

--- 2. GPIO PIN CONFIGURATION ---

```
GPIO.setmode(GPIO.BCM) # Configures the code to use the Broadcom (BCM) pin numbering scheme on the header.
```

Pin Definitions

```
TRIG_PIN = 17 # Assigns integer 17 to variable TRIG_PIN, representing the Trigger pin of the ultrasonic sensor.
ECHO_PIN = 27 # Assigns integer 27 to variable ECHO_PIN, representing the Echo pin of the ultrasonic sensor.
LED_GREEN = 14 # Assigns integer 14 to variable LED_GREEN, which is used to indicate forward movement.
LED_RED = 15 # Assigns integer 15 to variable LED_RED, which is used for the proximity/danger alert.
```

Pin Directions

```
GPIO.setup(TRIG_PIN, GPIO.OUT) # Sets the Trigger pin as an output so the Pi can send a signal out to the sensor.
GPIO.setup(ECHO_PIN, GPIO.IN) # Sets the Echo pin as an input so the Pi can listen for the sound bounce-back.
GPIO.setup(LED_GREEN, GPIO.OUT) # Sets the Green LED pin as an output to allow the Pi to turn the LED ON or OFF.
GPIO.setup(LED_RED, GPIO.OUT) # Sets the Red LED pin as an output to allow the Pi to turn the warning light ON or OFF.
```

--- 3. GLOBAL VARIABLES ---

```
# These variables store the "memory" of our standalone system.
```

```
global vehicle_state # Declares vehicle_state as global so all functions/threads can see its contents.
vehicle_state = "idle" # Initializes the state to "idle", meaning the vehicle stays stopped at startup.
```

```
global current_distance # Declares current_distance as global so sensor data is available to all logic blocks.
current_distance = 100.0 # Starts the system with a default distance of 100cm to avoid errors during initial boot.
```

--- 4. ULTRASONIC SENSOR LOGIC (CORE 1: THE WATCHDOG) ---

```
def monitor_obstacles(): # Begins the definition of the obstacle monitoring function.
```

```
    """This function runs forever in the background checking distance."""
```

```
    global current_distance # Allows this function to write data into the global distance variable.
```

```
    global vehicle_state # Allows this function to change the car's state (essential for autonomy).
```

```
    while True: # Starts an infinite loop that constantly cycles through distance measurements.
```

```
        # Trigger the sensor
```

```
        GPIO.output(TRIG_PIN, True) # Sends a HIGH signal to the Trigger pin to tell the sensor to send a sound wave.
```

```
        time.sleep(0.00001) # Keeps the trigger HIGH for exactly 10 microseconds as per HC-SR04 requirements.
```

```
        GPIO.output(TRIG_PIN, False) # Sets the Trigger pin to LOW to finish the start signal.
```

```

start_time = time.time() # Creates a timestamp for the exact moment the Trigger pulse finished.
stop_time = time.time() # Initializes a timestamp variable for when the sound returns.

# Listen for the Echo signal bounce-back
while GPIO.input(ECHO_PIN) == 0: # Checks if the Echo pin is still LOW (waiting for sound to start returning).
    start_time = time.time() # Updates start_time until the sound wave actually begins to reflect back.
while GPIO.input(ECHO_PIN) == 1: # Monitors when the Echo pin is HIGH (the pulse is returning).
    stop_time = time.time() # Records the final timestamp the moment the returning sound wave disappears.

# Calculation based on sound speed
duration = stop_time - start_time # Calculates the total travel time of the sound in seconds.
current_distance = (duration * 34300) / 2 # Multiplies time by speed of sound (34300cm/s) and divides by 2.

# CRITICAL STANDALONE LOGIC:
# Check if an obstacle is within the danger zone.
if current_distance < 25 and vehicle_state == "forward": # Checks if car is moving and an object is inside 25cm.
    print(f"!!! STANDALONE OVERRIDE: Obstacle at {current_distance}cm !!!") # Prints alert to local console.
    vehicle_state = "avoiding" # Instantly forces state to "avoiding" to stop the vehicle motion loop.

time.sleep(0.1) # Pauses the thread for 100ms to allow 10 precise readings per second.

# --- 5. HARDWARE EXECUTION LOOP ---
def run_hardware(): # Defines the function that will actually switch the LEDs based on the vehicle state.
    """This function turns states into real-world hardware responses."""
    global vehicle_state # Allows this function to read the current state set by either the sensor or the user.

    while True: # Starts an infinite loop to keep checking the state and updating LEDs.
        if vehicle_state == "forward": # Enters this branch if the user has successfully set the car to "forward".
            # If motion is authorized, green light stays ON.
            GPIO.output(LED_GREEN, GPIO.HIGH) # Supplies power to the Green LED to signal forward activity.
            GPIO.output(LED_RED, GPIO.LOW) # Cuts power to the Red LED as there is no obstacle danger.

        elif vehicle_state == "avoiding": # Enters this branch if the sensor watchdog detects a collision risk.
            # If an obstacle was detected, lights swap instantly.
            GPIO.output(LED_GREEN, GPIO.LOW) # Turns Green LED OFF because motion has been blocked.
            GPIO.output(LED_RED, GPIO.HIGH) # Turns Red LED ON to visually signal the autonomous intervention.

        else: # Default branch if the state is "idle" or anything else.
            # Idle/Stop mode
            GPIO.output(LED_GREEN, GPIO.LOW) # Keeps Green LED OFF because no move command is active.
            GPIO.output(LED_RED, GPIO.LOW) # Keeps Red LED OFF because there is no immediate collision alarm.

        time.sleep(0.1) # Waits 100ms to ensure the hardware isn't cycling unnecessarily fast.

# --- 6. FLASK WEB INTERFACE (REMOTE CONTROL) ---
app = Flask(__name__) # Creates an instance of the Flask class to build our web application.

```

```

@app.route("/control") # Creates a specific URL path at "/control" to receive input from mobile browsers.
def receive_command(): # Defines the function to be executed whenever someone visits the "/control" URL.
    global vehicle_state # Accesses global variable so user commands can set the state of Core 0.

    # Extract the command text from the mobile URL (e.g., cmd=forward)
    cmd = request.args.get('cmd', "").lower() # Reads value after "cmd=" and turns it into lowercase text.

    if "forward" in cmd: # Logic to process a "forward" request from the mobile device.
        # Check if the path is already clear before even trying to start.
        if current_distance > 30: # Only allows movement if the sensors see at least 30cm of empty space.
            vehicle_state = "forward" # Formally changes global state to forward motion.
            return jsonify({"status": "Moving Forward"}) # Sends a JSON confirmation message back to the phone.
        else: # Occurs if user tries to move "forward" but something is already blocking the car.
            return jsonify({"status": "Error: Path Blocked"}) # Warns the phone user that motion was denied.

    elif "stop" in cmd: # Logic to process an intentional "stop" request from the mobile user.
        vehicle_state = "idle" # Resets the global state to idle/waiting mode.
        return jsonify({"status": "Stopping"}) # Confirms the stop command to the mobile user.

    return jsonify({"status": "Unknown Command"}) # Default response if the phone user sends invalid text.

# --- 7. MAIN EXECUTION ---
if __name__ == "__main__": # Standard Python syntax to ensure this script is the one directly executed.
    # Start the standalone threads
    _thread.start_new_thread(monitor_obstacles, ()) # Fires up Core 1 exclusively to monitor sensors.
    _thread.start_new_thread(run_hardware, ()) # Fires up a dedicated loop exclusively to handle LED output.

    # Start the web server for mobile access
    # Port 80 allows easy access without extra numbers in the IP.
    app.run(host='0.0.0.0', port=80) # Launches the network server so any phone on the network can connect.

```

5.0 STEP-BY-STEP USER INSTRUCTIONS

Phase 1: Setting up the Raspberry Pi 5

1. **Imager:** On your laptop, open **Raspberry Pi Imager**.
2. **OS Selection:** Choose **Raspberry Pi OS (64-bit)**.
3. **Settings:** Click the cogwheel. Set a username (e.g., student) and password. **Enable SSH** and set your WiFi SSID/Password.
4. **Flash:** Write the OS to the microSD card and insert it into the Pi 5.
5. **Identify IP:** Connect your Pi to your monitor or use a network scan to find its local IP (Example: 192.168.1.50).

Phase 2: Preparing the AI Code

1. **Open Terminal:** SSH into your Pi from your laptop.
2. **Install Flask:** Type pip install flask in the terminal.
3. **Save Script:** Copy the code provided in Section 4.0 into a file named smart_car.py.
4. **Launch:** Run the script: sudo python3 smart_car.py.

Phase 3: Interaction through Mobile Device

1. **Connect:** Ensure your phone is on the **same WiFi** as the Raspberry Pi.
2. **Command via Voice:**

- Open your phone's browser (Safari/Chrome).
 - In the URL bar, tap the **Microphone Icon** on your phone's keyboard.
 - Say **"Forward"**.
 - Format the URL as: `http://192.168.1.50/control?cmd=forward`. (Your Pi's IP)
 - **Observe:** The Green LED turns ON. The car is "moving."
3. **Autonomous Event:**
- While the vehicle is "moving" (Green LED ON), wave your hand 15cm from the sensor.
 - **Result:** Instantly, the Red LED turns ON. The system is standalone; it didn't wait for your phone to tell it to stop, it protected itself.

Important: Explaining the specific code line (from flask import Flask, request, jsonify)

This line installs a 'Digital Receptionist' at the front door of your Raspberry Pi.

- **Flask (The Receptionist):** This is the main engine. It lets the Pi listen for "knocks" on its network address. When a phone "knocks" at the web address, Flask opens the door and executes the code associated with that path.
- **request (The Order Form):** When someone visits the Pi's web address, they usually bring info with them (like saying `cmd=forward`). `request` is the library we use to read that information. It's like the receptionist reading an order form left by a customer.
- **jsonify (The Translator):** Mobile phones and computers talk in a format called JSON. Python talks in "dictionaries." `jsonify` translates our Python code back into JSON so the mobile phone's browser can display a professional confirmation message like `{"status": "Moving Forward"}`.

How is the AI Model accessed through the mobile phone?

This is a **4-step process** that links the smartphone directly to the state machine we built in the Python code:

Step 1: The Network Connection (Same "Wifi" logic): The smartphone and the Raspberry Pi must be on the **same WiFi network**. The phone doesn't "talk" to the chip directly; it talks through the **Router**.

- The Pi has an IP address (e.g., 192.168.1.50).
- The Flask server is "sitting" on that IP address on **Port 80**.

Step 2: The HTTP Trigger: The student opens the phone's browser and types a specific address (a URL):
`http://192.168.1.50/control?cmd=forward`

- When the student hits **"Enter"** (or speaks into the phone's voice-to-text), the phone sends a request across the WiFi.

Step 3: Flask Hands Data to the AI logic: On the Pi, the line `@app.route("/control")` catches that request.

1. Flask uses `request` to see that `cmd` is equal to "forward".
2. It then calls the `receive_command()` function.
3. Inside that function, it changes the **Global Variable** `vehicle_state` to "forward".
4. **This is where the AI kicks in:**
 - The `monitor_obstacles` loop (Core 1) is already running.
 - The second it sees the `vehicle_state` change to "forward", it checks if `current_distance` is clear.
 - If the AI sensor logic approves, the car "moves."

Step 4: The Closed Feedback Loop: Once the logic is executed, `jsonify` sends a reply back to the phone. The phone screen will instantly update to show "Moving Forward".

Summary:

COMPONENT	ROLE IN THE AI ASSISTANT
MOBILE PHONE	The Input Interface . User provides instructions (Voice or Text).
FLASK (REQUEST)	The Courier . It carries instructions across the network to the Pi.
STATE VARIABLE	The Coordinator . It tells the logic what the user <i>wants</i> to do.
AI SENSOR (WATCHDOG)	The Governor . It looks at the world and decides if the command is <i>safe</i> to do.

The AI model isn't "on" the phone; it's "on" the Pi. The phone is simply a **window** through which we look at the brain of the assistant and tell it what we need.